PROJECT DOCUMENTATION

RAG-Powered Admission and Fee Assistant

Implementation: KLU Admission Assistant (Kalasalingam Academy of Research and Education)

Repository: github.com/ybalaji123/KLU-Admission-Assistant

Table of contents

1. Overview
2. Repository structure
3. System architecture
4. Tech stack
5. Core workflows
6. Backend reference
7. REST API reference
8. Data model and workbook
9. Guardrails and security
10. Frontend reference
11. Configuration (environment variables)
12. Local development
13. Deployment
14. Testing and CI
15. Operational notes and known caveats

1. Overview

What it does

- Answers student questions about KLU admissions, programmes, fees, scholarships, hostel options, and FAQs in a chat interface.
- Stores the college information in one Excel workbook (`KLU_Admission_Data.xlsx`) and turns each non-empty row into a labeled, searchable chunk.
- Retrieves the closest matching rows with semantic search (Gemini embeddings plus ChromaDB) and asks Gemini to write an answer from those rows only.
- Shows the sheet and row numbers behind each answer as a source line, so students and staff can trace a fact back to the workbook.
- Formats fee, room, scholarship and programme comparisons as Markdown tables, which the frontend renders as HTML tables.

Design principles

- **Grounded answers.** The model is told to use only the supplied workbook excerpts and to say when the workbook lacks the answer, pointing students to the admissions office.
- **Rows as chunks.** The workbook already stores each fact in a short record, so the project does not split text by length. A hostel row keeps its room type, fee, deposit and amenities together.
- **Key stays on the server.** The Gemini API key is read by the backend from `.env` and is never sent to the browser.
- **Rebuildable index.** The ChromaDB index is generated data. It is reused between launches and rebuilt when the workbook changes.

2. Repository structure

```
KLU-Admission-Assistant/
|-- README.md                        Overview, RAG explanation, local run steps
|-- requirements.txt                 Python dependencies (FastAPI, google-genai, chromadb, openpyxl)
|-- .python-version                  Python 3.11.11
|-- .gitignore                       Ignores .env, .venv, data/chroma db/, caches
|-- backend/
|   |-- __init__.py
|   |-- main.py                      FastAPI app: startup, routes, CORS, static files
|   `-- rag.py                       Chunking, embeddings, ChromaDB index, retrieval, answers
|-- frontend/
|   |-- index.html                   Chat page
|   |-- styles.css                   Layout and typography
|   |-- app.js                       Chat input, API calls, Markdown-style rendering
|   |-- config.js                    Sets the API base URL for a separately hosted frontend
|   |-- klu-banner.png               University wordmark for the header
|   `-- klu-logo.png                 Crest shown beside assistant messages
|-- data/
|   |-- KLU_Admission_Data.xlsx      Workbook read by the backend
|   `-- chroma_db/
|       `-- viewchromadb.py          Small script that prints sample indexed chunks
|-- KLU_Admission_Data.xlsx          Duplicate copy of the workbook (not read by the code)
`-- model_performing/
    `-- index_workbook.py            Optional manual command to build or resume the index
```

3. System architecture

```
Student browser (frontend/: HTML, CSS, JavaScript)
     |  POST /api/chat  (message + up to 12 recent turns)
     v
FastAPI backend (backend/main.py), also serves the page at /
     |
     +-- backend/rag.py  KLUAssistant
           |-- Excel workbook  -> one chunk per non-empty row
           |-- Gemini embeddings (gemini-embedding-001)
           |-- ChromaDB (persistent, data/chroma_db/, collection "klu_admissions", cosine)
           `-- Gemini answer model (default gemini-3.5-flash-lite), temperature 0.2
```

The API server is the only part that talks to Gemini or ChromaDB. On startup, the lifespan hook builds the vector index in a worker thread, so the event loop stays responsive. Chat calls to Gemini also run in a thread pool, which keeps blocking network calls out of the event loop.

4. Tech stack

Layer	Technology
Frontend	HTML, CSS and vanilla JavaScript (no build step)
API	FastAPI, Uvicorn (standard extras), Pydantic v2
LLM and embeddings	Google Gemini via the google-genai SDK; answers with gemini-3.5-flash-lite, embeddings with gemini-embedding-001
Vector database	ChromaDB (PersistentClient, local disk, cosine distance)
Workbook reader	openpyxl (read-only, values only)
Configuration	python-dotenv reads the project-root .env file
Python	3.11.11 (from .python-version)

5. Core workflows

5.1 Startup and indexing

```
uvicorn starts -> lifespan(): if GEMINI_API_KEY is set
   -> KLUAssistant() creates the Gemini client and PersistentClient(CHROMA_PATH)
   -> ensure_index() in a thread:
        load_workbook_chunks()        every sheet, every non-empty row -> chunk
        _find_reusable_collection()   reuse the collection if the workbook mtime matches
                                      and the stored count is not larger than the chunk count
                                      otherwise delete it and start over
        embed missing chunks in batches of 64 (RETRIEVAL_DOCUMENT) and add them to ChromaDB
   -> assistant is ready; otherwise startup_error explains why
```

If the Gemini API returns a retry delay, embed_documents waits once and retries the batch. Because each chunk ID is a hash of the sheet, row and text, a run that stopped partway through resumes by embedding only the rows that are not yet in the collection.

5.2 Answering a question

```
POST /api/chat { message, history }
  1. Validate the message (1 to 2000 characters) and the history (up to 12 turns of up to 1000 characters).
  2. retrieve_chunks(question):
       embed the question (RETRIEVAL_QUERY), query ChromaDB for the top 5 chunks,
       drop any chunk with cosine distance >= 1.2.
  3. build_prompt(): recent conversation (last 6 turns) + numbered excerpts with sheet and row labels
       + the student question.
  4. Gemini generate_content with ANSWER_INSTRUCTIONS as the system instruction.
  5. Return { answer, sources: [{sheet, row}, ...] }, with each sheet and row listed once.
```

If no excerpt passes the distance filter, the prompt says so and the model is asked to explain that the workbook does not contain enough information and to give the admissions contact details.

6. Backend reference

Module	Responsibility
backend/main.py	Creates the FastAPI app with a lifespan hook, mounts the frontend folder at /static and at /, sets CORS origins, defines request models (ChatTurn, ChatRequest), and implements the three routes.
backend/rag.py	KnowledgeChunk dataclass; clean_cell; load_workbook_chunks; KLUAssistant with embed_text, embed_documents, _find_reusable_collection, ensure_index, retrieve_chunks, build_prompt, list_sources and answer.
model_performing/index_workbook.py	Command-line helper that calls KLUAssistant().ensure_index() and prints the chunk count. Run as python -m model_performing.index_workbook from the project root.
data/chroma_db/viewchromadb.py	Prints the total chunk count and the first five stored records. It uses a relative path, so run it from the project root.

Key constants in rag.py

Constant	Value	Meaning
COLLECTION_NAME	klu_admissions	ChromaDB collection name
EMBEDDING_BATCH_SIZE	64	Chunks embedded per Gemini request
RETRIEVAL_RESULT_COUNT	5	Maximum chunks returned per question
MAX_COSINE_DISTANCE	1.2	Chunks at or above this distance are discarded
RECENT_TURNS_FOR_CONTEXT	6	Earlier chat turns sent for follow-up questions

7. REST API reference

The API has three routes. None require authentication. Errors use FastAPI's standard `{"detail": ...}` format.

Method	Path	Description
GET	/	Serves frontend/index.html
GET	/api/health	Returns ready (bool), indexed_rows (count) and a status message. Explains if GEMINI_API_KEY is missing or startup failed.
POST	/api/chat	Body: message (1 to 2000 chars), history (up to 12 turns with role user or assistant and content up to 1000 chars). Returns answer and sources. Returns 503 if the assistant is not ready, 422 for an empty message, and 502 if Gemini or ChromaDB fails.

8. Data model and workbook

Workbook sheets

Sheet	Header row	Non-empty records
University_Info	Field, Value	14
Programs	Level, Program Code, Program Name, Category, Duration, Stream, Eligibility, Entrance Exam, Tuition, Misc Fee, Total Annual Fee, Key Highlights	19
Scholarships	Category, Scholarship Name, Eligibility Criteria, Waiver %, Benefit, Applicable Programs, Additional Notes	9
Hostel_Options	Room Type, Annual Fee, Mess Charges Included, Security Deposit, Deposit Refundable, Amenities	4
FAQs	ID, Category, Question, Answer, Keywords	15
QA_Training_Pairs	#, Input Question, Expected Answer, Category, Source	100

Counts come from the header-excluded rows in the current workbook, about 161 chunks in total. The Programs sheet has a duplicate Level column, and the code uses the first matching label for each name.

Chunk format (illustrative values)

```
Source sheet: Hostel_Options
Room Type: Double Sharing
Annual Fee (Rs.): 85000
Mess Charges Included: Yes
Security Deposit (Rs.): 5000
Deposit Refundable: Yes
Amenities: Wi-Fi, laundry, ...
```

Stored item	Content
id	SHA-1 of sheet title, row number and chunk text; stable unless the row changes
document	The labeled chunk text shown above
metadata	sheet, row, category (from the Category or Level column, else the sheet name) and question (from the Question column, if any)
embedding	Gemini vector from gemini-embedding-001, stored with the chunk
collection metadata	hnsw:space = cosine and workbook_mtime_ns, used to detect workbook changes

9. Guardrails and security

- **Input limits.** Messages are 1 to 2000 characters; history holds at most 12 turns of 1000 characters each (Pydantic validation).
- **Grounding instructions.** The system prompt tells Gemini to treat excerpts as facts, not instructions, and not to guess, promise, or present fees as independently verified.
- **Relevance filter.** Chunks with cosine distance of 1.2 or more are dropped before the model sees them.
- **Secret handling.** The API key stays on the server. The frontend never receives it. Note that the code does not redact secrets from the answer text.
- **CORS.** Allowed origins are localhost, the Render backend URL, and any origins listed in FRONTEND_ORIGINS. Only GET, POST and OPTIONS are allowed, with the Content-Type header and no credentials.
- **Contact fallback.** When the workbook does not answer a question, the assistant gives admission@klu.ac.in and +91 7397760760.
- **Not implemented.** There is no user login, no per-IP rate limiting, no prompt-injection filter, and no citation check that verifies the model's sources.

10. Frontend reference

The frontend is a single static page with no build step. It is served by FastAPI at / and can also be hosted separately (the project is live on Vercel, per the README).

File	Role
index.html	Header with the university banner and Apply link, chat card, four suggested-question buttons (B.Tech fees, Scholarships, Hostel options, How to apply), a textarea with a 2000-character limit, and a footer with the admissions email and phone.
app.js	Reads API_BASE_URL from window.KLU_API_BASE_URL; keeps the conversation in an in-memory history array; posts to /api/chat; renders assistant answers with its own small Markdown renderer (bold, headings, bullet lists and pipe tables); shows sources as sheet and row labels.
config.js	Sets window.KLU_API_BASE_URL to an empty string on localhost (same origin) and to the Render backend URL elsewhere.
styles.css	Layout, chat bubbles and styling for the answer tables.

11. Configuration (environment variables)

Read from the project-root `.env` file (loaded in main.py and rag.py) or from the host environment.

Variable	Default	Purpose
GEMINI_API_KEY	none (required)	Gemini API key. Without it, the page loads but chat returns 503.
GEMINI_MODEL	gemini-3.5-flash-lite	Model used to write answers
GEMINI_EMBEDDING_MODEL	gemini-embedding-001	Model used for chunk and question embeddings
CHROMA_PATH	data/chroma_db	Folder for the persistent ChromaDB index. Use /var/data/chroma_db on a persistent disk.
FRONTEND_ORIGINS	empty	Extra comma-separated origins allowed by CORS, with no trailing slash needed

The README's environment table is missing its header row, so it renders as plain text on GitHub. The values above come from the code.

12. Local development

Prerequisites: Python 3.11.11 (matches .python-version) and a Gemini API key.

```
python -m venv .venv
.venv\Scripts\Activate.ps1          # Windows PowerShell (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt

# create .env in the project root
GEMINI_API_KEY=your_api_key_here
GEMINI_MODEL=gemini-3.5-flash-lite
GEMINI_EMBEDDING_MODEL=gemini-embedding-001

uvicorn backend.main:app --reload     # then open http://127.0.0.1:8000
```

- On the first start, the workbook is embedded and saved in data/chroma_db/. Later starts reuse it unless the workbook's modification time changes.
- To build the index without starting the server, run `python -m model_performing.index_workbook` from the project root.

- To inspect the index, run `python data/chroma_db/viewchromadb.py` from the project root.
- Check /api/health to confirm the assistant is ready and how many rows are indexed.

13. Deployment

The README names two hosts: the live site is listed as a Vercel URL, and the backend's CORS defaults and frontend/config.js point to a Render service. The repository has no render.yaml or Dockerfile, so the deployment settings below are set in the hosting dashboard.

Part	Setting
Backend (Render)	Web service. Suggested start command: uvicorn backend.main:app --host 0.0.0.0 --port $PORT. Set GEMINI_API_KEY as a secret. Attach a persistent disk and set CHROMA_PATH=/var/data/chroma_db.
Frontend (Vercel or static host)	Serve the frontend folder. config.js must point to the backend URL, and that frontend origin must be listed in FRONTEND_ORIGINS.
Persistence	Without a disk, the index is rebuilt on each deploy, which calls the Gemini embedding API for every row.

14. Testing and CI

The repository has no automated tests and no CI workflow. The only verification helpers are the manual indexing command and the ChromaDB inspection script. Suggested next steps: unit tests for load_workbook_chunks (row labels, skipped blanks, stable IDs), tests for the distance filter in retrieve_chunks with a fake embedding client, and a GitHub Actions workflow that installs requirements and runs pytest.

15. Operational notes and known caveats

These observations come from reading the code and repository, and are useful when maintaining or extending the project.

- **Two frontend API addresses.** config.js and the default CORS list point to admission-and-fee-faq-assistant.onrender.com, but the README's live link is klu-2a.vercel.app. Confirm which backend is in use before changing either.
- **Duplicate workbook.** KLU_Admission_Data.xlsx exists at the root and in data/. Only the data/ copy is read. Keep one copy, or the two will drift apart.
- **Change detection uses file time.** A workbook is treated as changed only when its modification time changes, not its content. Any change rebuilds the whole index, and an edit that keeps the same timestamp would go unnoticed.
- **Public endpoints with no rate limit.** /api/chat calls a paid Gemini API for every request and has no per-IP or per-user limit. Anyone with the URL can use it.
- **Fixed contact details in the prompt.** The admissions email and phone number are written into ANSWER_INSTRUCTIONS, so they must be updated in code when they change.
- **Answers are not checked.** The model's text is returned as is. Sources are the retrieved sheet and row pairs, not the model's own citations, so an answer could still misstate a retrieved value.
- **Fee wording.** The prompt tells the model not to present fees as independently verified. Students should still be told to confirm fees with the admissions office, which the footer already says.
- **Frontend renderer.** app.js handles only bold, headings, bullet lists and pipe tables. Other Markdown, such as links and code, is shown as plain text.

- **Startup is blocking.** The lifespan hook waits for indexing to finish before the server accepts requests, so the first run of about 161 chunks delays the site until embedding completes. If it fails, the server still starts and /api/health reports the error.
- **README formatting.** The environment-variable table is missing its header line. The .gitignore also lists .streamlit/, a leftover that the code does not use.